Juice Bottling

AlgoExpert · Hard · Dynamic Programming

The problem

- Prices[i] is the price of a bottle of size i (prices[0] = 0).
- Split the total (prices.length - 1 units) into bottle sizes maximising revenue; return sizes (rod cutting).

Intuition

- Unbounded knapsack over sizes: best price for each total is a bottle plus the best for the remainder.

Brute force vs optimal

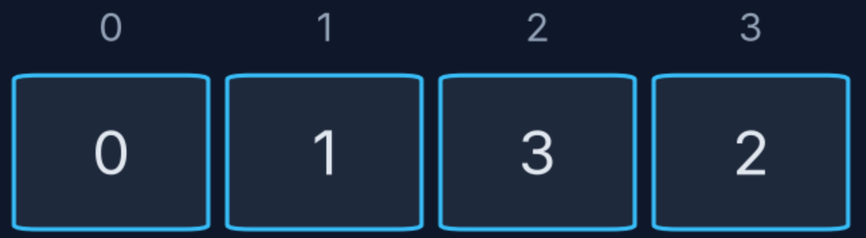
- Try all partitions → exponential
- Rod-cutting DP → $O(n^2)$
- Store the first cut to rebuild

Approach

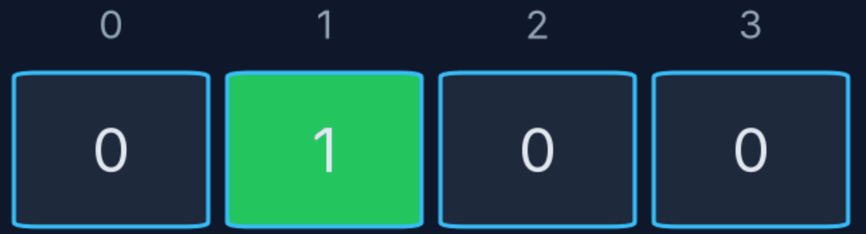
- 1. Best of zero is zero
- 2. For each size, try every first bottle
- 3. Store the best choice
- 4. Rebuild the bottle sizes

Worked example

price per bottle size



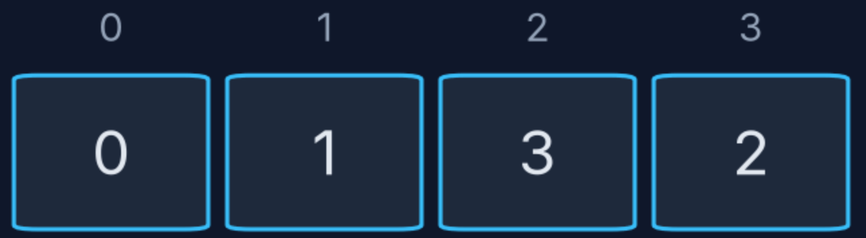
best revenue for amount



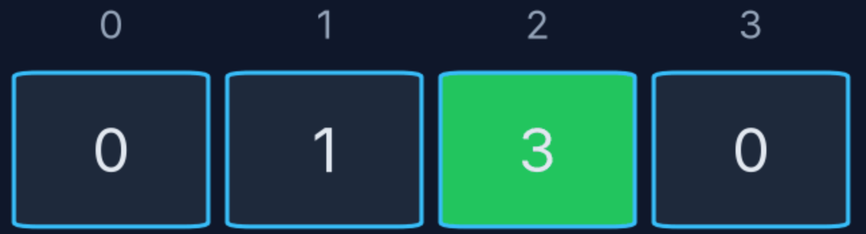
Amount 1: best 1 (first bottle 1)

Worked example

price per bottle size



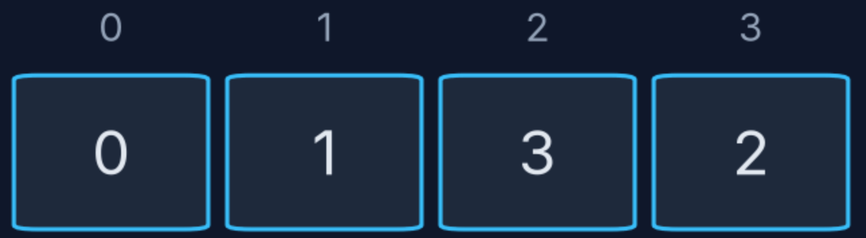
best revenue for amount



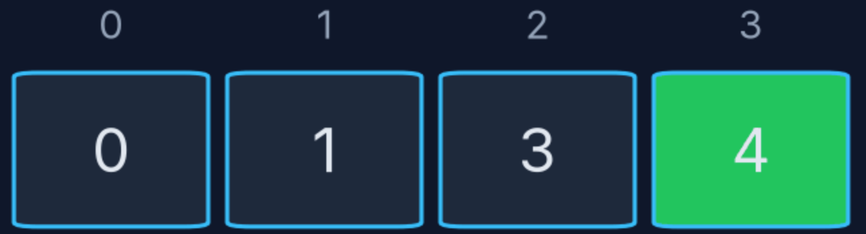
Amount 2: best 3 (first bottle 2)

Worked example

price per bottle size



best revenue for amount



Amount 3: best 4 (first bottle 1)

Worked example

```
bottles: [1, 2]

revenue 4
```

Answer: [1, 2]

Java solution

```java
public class JuiceBottling {
    public static List<Integer> juiceBottling(int[] prices) {
        int n = prices.length;
        int[] best = new int[n], cut = new int[n];
        for (int s = 1; s < n; s++)
            for (int k = 1; k <= s; k++)
                if (prices[k] + best[s - k] > best[s]) { best[s] = prices[k] + best[s - k]; cut[s] = k; }
        List<Integer> out = new ArrayList<>();
        for (int s = n - 1; s > 0; s -= cut[s]) out.add(cut[s]);
        return out;
    }
}
```

Complexity

- Time: O(n^2)
- Space: O(n)

Edge cases & pitfalls

- Prices[0] = 0
- Rebuild from cut[]
- Ties: any order fine

Interview tips & follow-ups

- Name it: rod cutting

Recap

- Unbounded knapsack over sizes: best price for each total is a bottle plus the best for the remainder.
- Complexity: O(n^2)